

Tech Mahindra - 100 Technical Interview Questions

Tech Mahindra

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. What is operator overloading? Give an example in C++.

Answer:

Operator overloading lets you redefine what an operator like '+' does for a user-defined type. For example, in C++ you could overload '+' for a Complex number class so that `c1 + c2` adds their real and imaginary parts, letting Complex objects be used with familiar arithmetic syntax.

Q2. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

Q3. What happens when a class implements an interface but does not implement all its methods?

Answer:

If a class implements an interface but doesn't implement all its abstract methods, the class itself must be declared abstract; otherwise, it is a compile-time error since an interface is a contract that guarantees every method has an implementation somewhere in the concrete class hierarchy.

Q4. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q5. Explain the difference between 'is-a' and 'has-a' relationships with examples.

Answer:

An 'is-a' relationship implies inheritance - a Dog is-a Animal. A 'has-a' relationship implies composition - a Car has-a Engine. Choosing 'has-a' over 'is-a' when there isn't a true specialization relationship avoids incorrectly forcing an inheritance hierarchy where composition would be more appropriate.

Q6. Explain the concept of a virtual function in C++ and how it enables runtime polymorphism.

Answer:

A virtual function in C++ is declared with the 'virtual' keyword in a base class, telling the compiler to resolve the call at runtime via a vtable based on the actual object type rather than the reference/pointer type. This is what allows a Shape* pointing to a Circle to correctly call Circle's draw() instead of Shape's - the mechanism behind runtime polymorphism in C++.

Q7. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q8. What is polymorphism? Differentiate between compile-time and runtime polymorphism with examples.

Answer:

Polymorphism lets the same interface behave differently depending on the actual object. Compile-time (static) polymorphism is resolved during compilation - method/operator overloading is the classic example. Runtime (dynamic) polymorphism is resolved during execution based on the actual object type - method overriding via a base-class reference is the classic example.

Q9. What is a functional interface in Java, and how does it relate to lambda expressions?

Answer:

A functional interface is an interface with exactly one abstract method (it may have default/static methods too), e.g. Runnable or Comparator. Java's lambda expressions are essentially shorthand implementations of a functional interface's single abstract method, letting you write (a, b) -> a.compareTo(b) instead of an anonymous inner class.

Q10. Explain the SOLID principles of object-oriented design.

Answer:

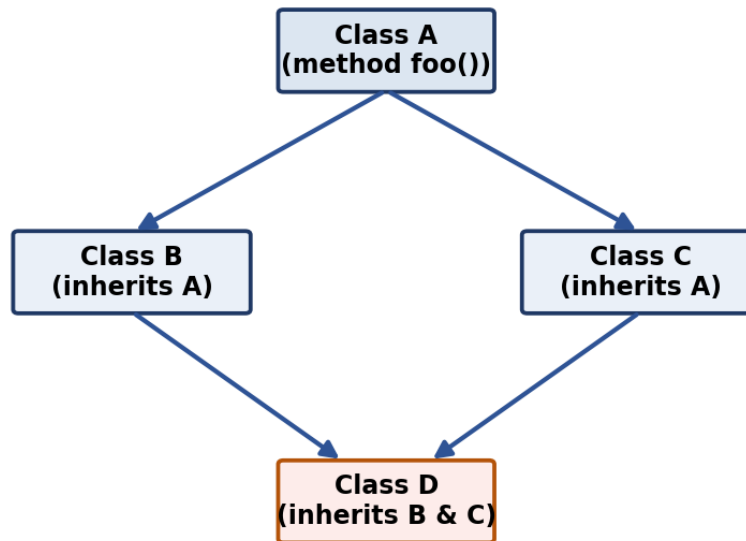
SOLID stands for: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable for their base types without breaking correctness), Interface Segregation (prefer many small specific interfaces over one large one), and Dependency Inversion (depend on abstractions, not concrete implementations).

Q11. What is the 'diamond problem' in multiple inheritance, and how does Java avoid it?

Answer:

The diamond problem arises when a class inherits from two classes that both inherit from a common base class, creating ambiguity about which inherited method version to use. Java avoids it for classes by disallowing multiple inheritance of classes entirely (a class can extend only one class), and for interfaces (Java 8+ default methods) it forces the implementing class to explicitly override the conflicting method if two interfaces provide the same default.

Diamond Problem: which foo() does D inherit - via B or via C?



Q12. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q13. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q14. What is object cloning, and what is the difference between a shallow copy and a deep copy?

Answer:

Object cloning creates a copy of an existing object. A shallow copy duplicates the object's top-level fields but nested objects/references are shared with the original (mutating a nested object affects both copies). A deep copy recursively duplicates nested objects too, so the copy is fully independent of the original.

DBMS & SQL

Q15. How does a B-Tree index improve query performance internally?

Answer:

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically $O(\log n)$) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q16. What is a stored procedure, and what are its advantages over writing ad-hoc queries?

Answer:

A stored procedure is a precompiled, named collection of SQL statements stored in the database and invoked as a single call. Advantages include reduced network round-trips (one call instead of many statements), better security (grant execute rights without exposing underlying tables directly), reusability across applications, and a cached execution plan that can improve performance over repeatedly sent ad-hoc queries.

Q17. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q18. Explain the difference between a primary key, a unique key, and a candidate key.

Answer:

A candidate key is any column (or set of columns) that could uniquely identify a row - a table can have several. A primary key is the one candidate key chosen to be the table's main unique identifier (cannot be NULL). A unique key is a constraint enforcing uniqueness on a column that is not the primary key, and unlike a primary key, it typically allows one NULL value.

Q19. What is a view in SQL, and how is it different from a materialized view?

Answer:

A view is a virtual table defined by a stored SELECT query - it has no data of its own and is recomputed each time it's queried. A materialized view physically stores the result set on disk and must be refreshed periodically (or on trigger), trading storage and staleness for much faster read access compared to recomputing a complex query every time.

Q20. What is the difference between DELETE, DROP, and TRUNCATE?

Answer:

DELETE removes rows one at a time (can be filtered with WHERE, is logged, and can be rolled back) and fires triggers; DROP removes the entire table structure and its data permanently; TRUNCATE quickly removes all rows by deallocating data pages (minimal logging), resets identity counters, but keeps the table structure - and cannot be rolled back in most databases once committed.

Q21. What is denormalization, and when would you intentionally denormalize a schema?

Answer:

Denormalization intentionally introduces redundancy (e.g. storing a customer's name directly on an Orders row instead of always joining to a Customers table) to reduce the number of joins needed and speed up read-heavy queries, at the cost of extra storage and more complex writes to keep the redundant copies in sync. It's used in reporting/analytics systems where read performance matters more than write simplicity.

Q22. Write a SQL query to find the Nth highest salary using a window function like RANK() or DENSE_RANK().

Answer:

SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM Employee) ranked WHERE rn = N (substituting the desired N). DENSE_RANK() assigns ranks without gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.

Q23. Write a SQL query using GROUP BY and HAVING to find departments with more than 5 employees.

Answer:

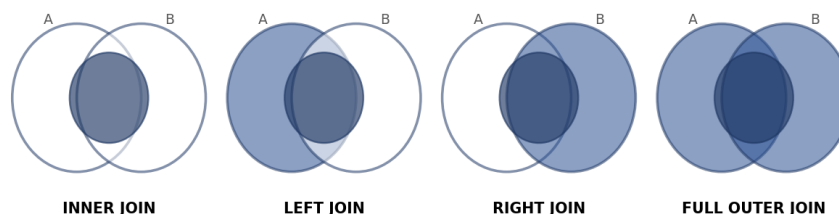
SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 5. GROUP BY collapses rows into per-department groups, and HAVING filters those groups (after aggregation) to keep only departments with more than five employees.

Q24. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN with examples.

Answer:

INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table plus matched rows from the right (NULLs where there's no match). RIGHT JOIN is the mirror - all rows from the right table plus matches from the left. FULL OUTER JOIN returns all rows from both tables, with NULLs wherever a match is missing on either side.

SQL JOIN Types (shaded = rows returned)



Q25. Write a SQL query to find duplicate records in a table.

Answer:

Using GROUP BY with HAVING COUNT(*) > 1: SELECT column_name, COUNT(*) FROM table_name GROUP BY column_name HAVING COUNT(*) > 1. This groups rows by the column(s) that define a duplicate and returns only the groups appearing more than once.

Q26. Write a SQL query to find the second-highest salary from an Employee table without using LIMIT/TOP.

Answer:

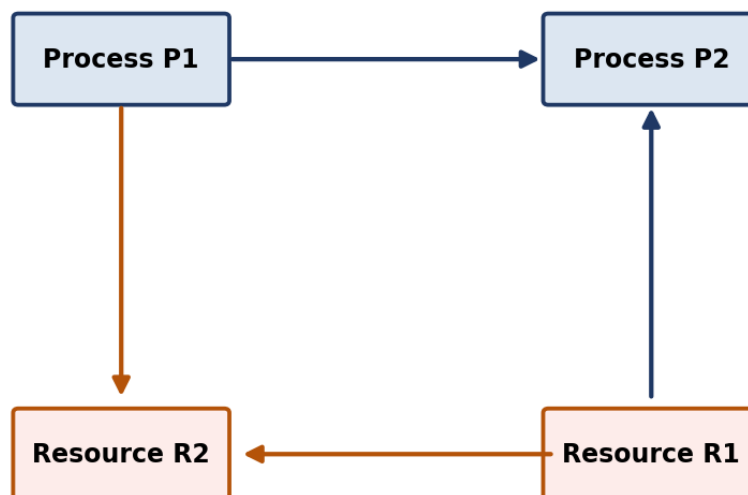
Using a subquery: `SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee)`. This first finds the overall maximum, then finds the maximum salary that is strictly less than it, giving the second-highest value without relying on database-specific LIMIT/TOP syntax.

Q27. What is a deadlock in a database, and how can it be prevented or resolved?

Answer:

A database deadlock happens when two or more transactions each hold a lock the other needs and wait on each other indefinitely (e.g. Transaction A locks Row 1 and waits for Row 2, while Transaction B locks Row 2 and waits for Row 1). It can be prevented by always acquiring locks in a consistent order across transactions, kept short by minimizing transaction scope, or resolved by the database's deadlock detector automatically rolling back one transaction (the 'victim') so the other can proceed.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q28. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q29. Explain database isolation levels and the anomalies (dirty read, non-repeatable read, phantom read) each one prevents.

Answer:

Isolation levels control how much one transaction can see of another's uncommitted work: Read Uncommitted allows dirty reads (seeing uncommitted changes); Read Committed prevents dirty reads but allows non-repeatable reads (a value changes between two reads in the same transaction); Repeatable Read prevents that but allows phantom reads (new rows appearing in a repeated range query); Serializable prevents all three by fully isolating transactions, at the cost of concurrency.

Q30. Explain the ACID properties of a database transaction with an example for each.

Answer:

ACID: **Atomicity** - a transaction's operations either all succeed or all fail together (e.g. a bank transfer's debit and credit both happen or neither does); **Consistency** - a transaction moves the database from one valid state to another, respecting constraints; **Isolation** - concurrent transactions don't see each other's incomplete/intermediate changes; **Durability** - once committed, changes survive a crash (written to persistent storage).

Q31. Explain the difference between a clustered and a non-clustered index.

Answer:

A clustered index determines the physical storage order of the table's data rows - a table can have only one, since data can only be sorted one way physically. A non-clustered index is a separate structure that stores pointers back to the actual data rows, so a table can have several, useful for speeding up queries on columns other than the clustering key.

Q32. Explain the difference between horizontal and vertical partitioning of a table.

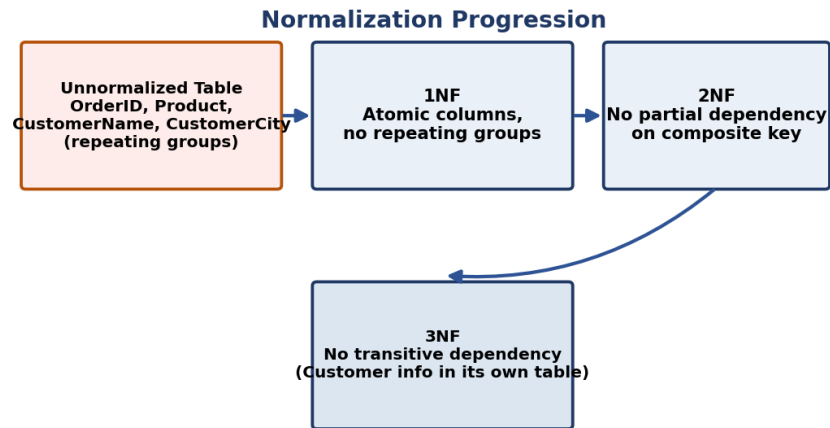
Answer:

Horizontal partitioning splits a table by rows (e.g. orders from 2023 in one partition, 2024 in another), similar in spirit to sharding but often within a single database instance. Vertical partitioning splits a table by columns (e.g. splitting rarely used large text/blob columns into a separate table), reducing the size of rows scanned for common queries that don't need those columns.

Q33. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with a practical example.

Answer:

Normalization removes redundancy step by step: **1NF** requires atomic column values with no repeating groups; **2NF** requires 1NF plus no partial dependency of a non-key column on part of a composite primary key; **3NF** requires 2NF plus no transitive dependency (a non-key column depending on another non-key column); **BCNF** is a stricter version of 3NF handling certain anomalies where a table has multiple overlapping candidate keys. Example: splitting an Orders table that repeats CustomerCity for every order into a separate Customers table removes the transitive dependency.



Q34. Explain how you would design a database schema for a simple e-commerce order system (tables and relationships).

Answer:

A minimal e-commerce schema would include: Customers (customer_id PK, name, email), Products (product_id PK, name, price), Orders (order_id PK, customer_id FK, order_date, status), and OrderItems (order_item_id PK, order_id FK, product_id FK, quantity, price_at_purchase) as a junction table capturing the many-to-many relationship between Orders and Products.

Operating Systems

Q35. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q36. What is the difference between a hard link and a symbolic link in a file system?

Answer:

A hard link is essentially another directory entry pointing directly to the same inode (and underlying data) as the original file - deleting the original name still leaves the data accessible via the hard link, and it cannot span file systems. A symbolic (soft) link is a small separate file that stores the path to the target file - it can point across file systems or to directories, but becomes a 'dangling link' if the target is deleted or moved.

Q37. What is a critical section, and what conditions must a solution to the critical-section problem satisfy?

Answer:

A critical section is a segment of code that accesses shared resources and must not be executed by more than one process/thread concurrently to avoid race conditions. A valid solution must guarantee: Mutual Exclusion (only one process in the critical section at a time), Progress (a process not in the critical section can't block others from entering), and Bounded Waiting (a limit on how many times other processes can enter before a waiting process gets its turn).

Q38. Compare FCFS, SJF, Round Robin, and Priority scheduling algorithms - what are the trade-offs of each?

Answer:

FCFS (First-Come-First-Served) is simple but can cause the 'convoy effect' where a long job delays all others. SJF (Shortest Job First) minimizes average waiting time but requires knowing job lengths in advance and can starve long jobs. Round Robin gives each process a fixed time quantum, ensuring fairness and responsiveness for interactive systems, but too small a quantum increases context-switch overhead. Priority Scheduling runs the highest-priority job first but can starve low-priority jobs unless aging is used.

Q39. Explain the difference between a monolithic kernel and a microkernel.

Answer:

A monolithic kernel runs all core OS services (file system, device drivers, memory management, scheduling) in a single address space in kernel mode, which is fast (no IPC overhead between components) but a bug in any component can crash the whole system. A microkernel keeps only the bare minimum (IPC, basic scheduling, memory management) in kernel space and moves other services to user-space processes, improving fault isolation and modularity at the cost of more inter-process communication overhead.

Q40. What is the difference between internal and external fragmentation?

Answer:

Internal fragmentation is wasted space inside an allocated fixed-size block that's larger than what the process actually needs (common in paging). External fragmentation is wasted space between allocated blocks - free memory exists in total but is scattered in pieces too small individually to satisfy a new request (common in segmentation/variable-size allocation).

Q41. What is a context switch, and why does it have a performance cost?

Answer:

A context switch is the process of saving the CPU state (registers, program counter) of a currently running process/thread and loading the saved state of another, so execution can resume later exactly where it left off. It has a performance cost because the CPU does no useful work during the switch, and it can also cause cache/TLB misses afterward as the new process's data isn't cached yet.

Q42. What is virtual memory, and why is it useful even when physical RAM is limited?

Answer:

Virtual memory gives each process the illusion of a large, contiguous address space by mapping logical addresses to physical RAM or disk (swap space) via page tables, transparently to the program. It's useful even with limited RAM because it allows programs larger than physical memory to run, isolates each process's memory from others for safety, and lets the OS overcommit memory across many processes that don't all need their full memory at once.

Q43. Explain the producer-consumer problem and how you would solve it using semaphores.

Answer:

The producer-consumer problem involves a producer adding items to a shared bounded buffer and a consumer removing them, needing synchronization so the producer doesn't add to a full buffer and the consumer doesn't remove from an empty one. It's solved using two counting semaphores (one tracking empty slots, one tracking filled slots) plus a mutex protecting the buffer itself during actual insert/remove operations.

Q44. Explain how a page fault is handled by the operating system.

Answer:

When a process accesses a page not currently in physical memory, the CPU raises a page fault trap. The OS then: locates the required page on disk, finds a free frame (or selects a victim page to evict via a page replacement algorithm like LRU, writing it back to disk if modified), loads the required page into that frame, updates the page table, and resumes the instruction that caused the fault.

Q45. Explain the difference between preemptive and non-preemptive scheduling with examples of each.

Answer:

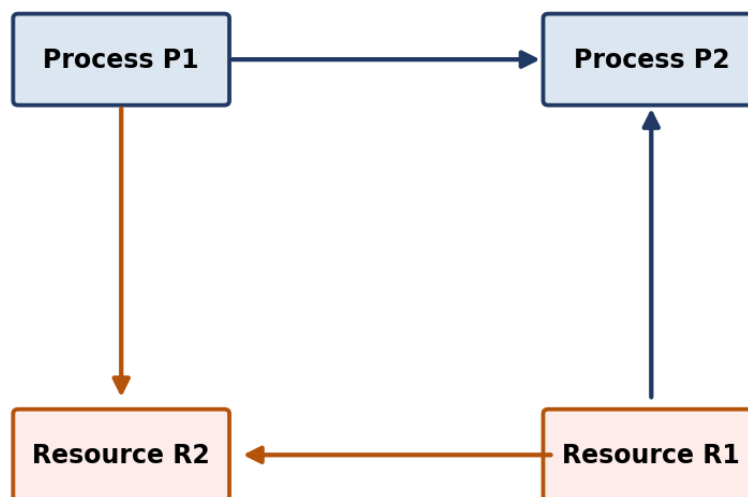
Preemptive scheduling allows the OS to forcibly suspend a running process to give the CPU to another (e.g. Round Robin, Priority Scheduling with preemption) - responsive but has context-switch overhead. Non-preemptive scheduling lets a running process keep the CPU until it voluntarily yields or finishes (e.g. FCFS, non-preemptive SJF) - simpler but a long process can block shorter ones ('convoy effect').

Q46. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.

Circular Wait: P1 -> R2 -> (held by P2) -> R1 -> (held by P1)



Q47. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Q48. What is a semaphore? Differentiate between binary and counting semaphores.

Answer:

A semaphore is an integer variable used for process synchronization, with atomic increment (signal/V) and decrement (wait/P) operations. A binary semaphore takes only values 0 or 1 and behaves like a mutex, ensuring exclusive access. A counting semaphore can take any non-negative integer, used to manage a pool of multiple identical resources (e.g. a fixed number of database connections).

Computer Networks

Q49. What is the difference between a proxy server and a reverse proxy?

Answer:

A proxy server sits between clients and the internet, forwarding client requests outward - useful for caching, content filtering, or anonymizing client identity from the destination server's perspective. A reverse proxy sits in front of one or more backend servers, forwarding client requests to them - useful for load balancing, SSL termination, and hiding backend server details from clients.

Q50. What is a CDN, and how does it reduce latency for end users?

Answer:

A CDN (Content Delivery Network) is a geographically distributed network of cache servers that store copies of static content (images, videos, scripts) closer to end users. When a user requests content, it's served from the nearest CDN edge server rather than the origin server, reducing round-trip latency and offloading traffic from the origin.

Q51. Explain the seven layers of the OSI model and the role of each layer.

Answer:

The seven OSI layers, bottom to top: **Physical** - raw bit transmission over a medium; **Data Link** - framing and MAC addressing between directly connected nodes; **Network** - logical addressing and routing between networks (IP); **Transport** - end-to-end reliable/unreliable delivery (TCP/UDP); **Session** - managing sessions/connections between applications; **Presentation** - data formatting, encryption, compression; **Application** - the actual protocols apps use (HTTP, FTP, DNS).

OSI Model - 7 Layers



Q52. What is ARP, and how does it map an IP address to a MAC address?

Answer:

ARP (Address Resolution Protocol) resolves a known IP address to its corresponding MAC address on a local network. When a device needs to send a frame to an IP it doesn't have a MAC address for, it broadcasts an ARP request ('who has this IP?'), and the device owning that IP responds with its MAC address, which the sender then caches for future use.

Q53. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Q54. Explain the difference between unicast, multicast, and broadcast communication.

Answer:

Unicast sends data from one sender to exactly one specific receiver (e.g. a normal web request). Multicast sends data from one sender to a specific group of interested receivers simultaneously (e.g. IPTV streaming to subscribed clients). Broadcast sends data from one sender to every device on the local network segment, regardless of interest (e.g. ARP requests, DHCP discovery).

Q55. Explain how SSL/TLS certificates establish trust between a client and a server.

Answer:

An SSL/TLS certificate, issued by a trusted Certificate Authority (CA), binds a public key to a domain's identity. During the TLS handshake, the server presents its certificate; the client verifies it was signed by a CA it trusts (via the certificate chain up to a trusted root CA), confirming the server is who it claims to be before establishing an encrypted session using keys derived during the handshake.

Q56. Explain congestion control mechanisms used by TCP (e.g., slow start, congestion avoidance).

Answer:

TCP congestion control prevents a sender from overwhelming the network. 'Slow start' begins with a small congestion window and doubles it each round-trip until reaching a threshold or detecting loss. 'Congestion avoidance' then increases the window more conservatively (linearly) to probe for more bandwidth without causing congestion. On packet loss, TCP reduces the window sharply (e.g. halving it) and restarts the process, adapting to network conditions.

Q57. What is the difference between a stateful and stateless protocol?

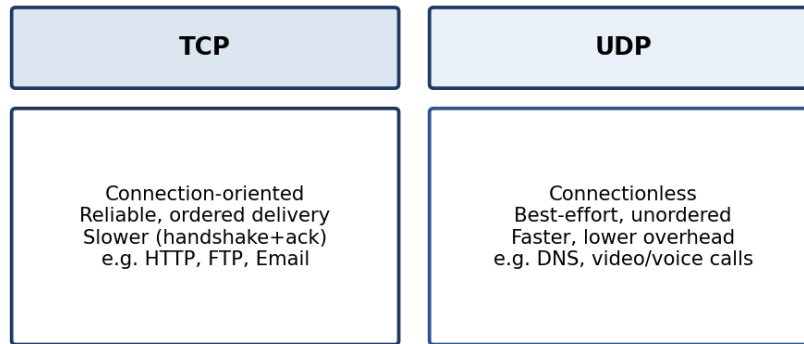
Answer:

A stateful protocol (like FTP or a typical database connection) requires the server to remember information about the client's session across multiple requests. A stateless protocol (like HTTP by default) treats every request independently, with no memory of previous requests - any state needed (like login status) must be carried explicitly in each request, e.g. via cookies or tokens.

Q58. What is the difference between TCP and UDP? Give an example use case for each.

Answer:

TCP is connection-oriented, establishes a session via a handshake, guarantees reliable and ordered delivery through acknowledgments and retransmission, but has more overhead - used for web pages, file transfers, and email. UDP is connectionless, sends packets with no delivery guarantee or ordering, but is faster with lower overhead - used for DNS lookups, video calls, and online gaming where speed matters more than occasional lost packets.

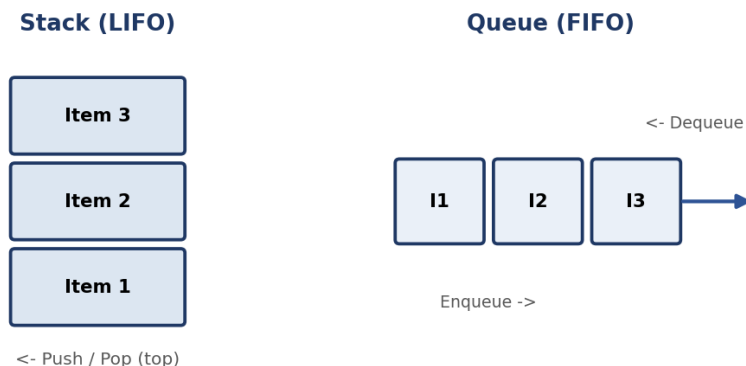


Data Structures & Algorithms

Q59. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q60. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

Answer:

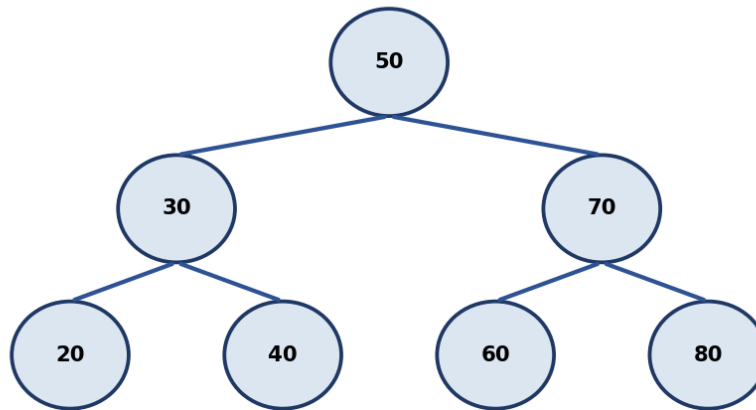
A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q61. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to $\log n$ by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing $O(\log n)$ search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to $O(n)$. Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q62. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q63. Explain how you would design an LRU cache and the data structures you'd use to achieve $O(1)$ get/put operations.

Answer:

An LRU cache needs $O(1)$ get and put operations. The standard design combines a hashmap (key $\rightarrow$ pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in $O(1)$; when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in $O(1)$, with the hashmap updated accordingly.



$O(1)$ get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q64. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q65. What is the sliding window technique, and in what type of problems is it useful?

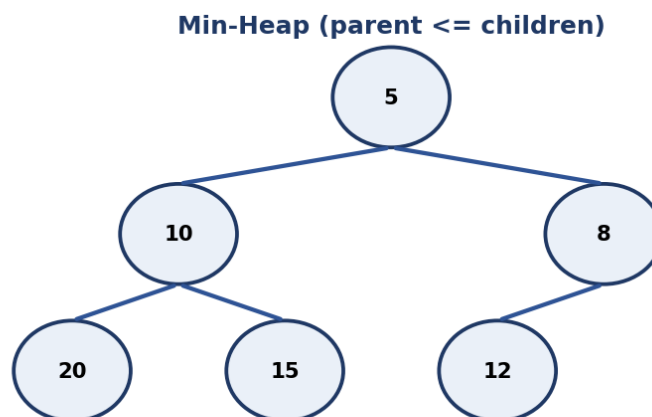
Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k, or longest substring without repeating characters), typically turning an $O(n^2)$ brute force into an $O(n)$ solution.

Q66. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take $O(\log n)$ since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q67. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

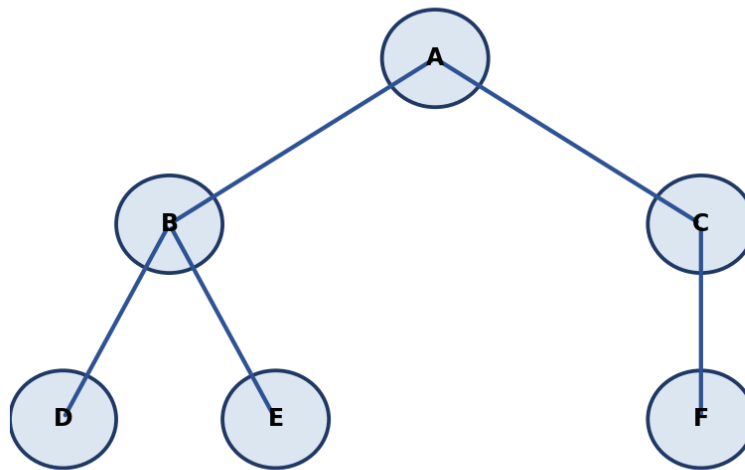
Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in $O(E \log V)$. Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q68. Explain the difference between BFS and DFS graph traversal, including their use cases.

Answer:

BFS (Breadth-First Search) explores a graph level by level using a queue, visiting all neighbors of a node before moving deeper - useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, using a stack (or recursion) - useful for tasks like detecting cycles, topological sorting, or exploring all possible paths/states.

Graph: BFS order A,B,C,D,E,F | DFS order A,B,D,E,C,F

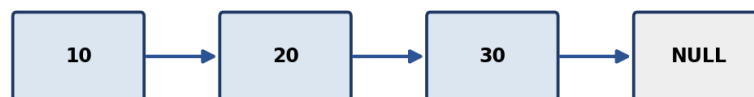


Q69. How would you detect a cycle in a linked list? Explain Floyd's cycle detection algorithm.

Answer:

Floyd's cycle detection (the 'tortoise and hare' algorithm) uses two pointers moving through the linked list at different speeds - one advances one node at a time (slow), the other advances two nodes at a time (fast). If there is a cycle, the fast pointer will eventually 'lap' the slow pointer and they will meet at the same node; if the fast pointer reaches the end (null), there is no cycle. This runs in $O(n)$ time and $O(1)$ space.

Singly Linked List

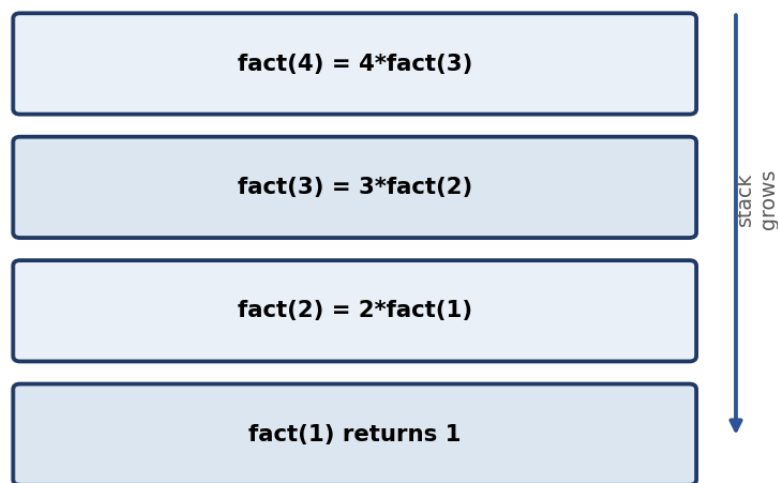


Q70. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.

Call Stack while computing factorial(4)

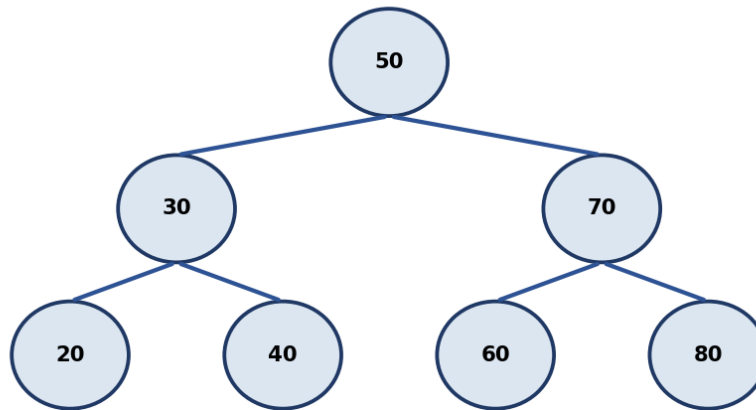


Q71. What is a binary search tree, and what is the time complexity of insertion, deletion, and search in the average and worst case?

Answer:

A Binary Search Tree (BST) is a tree where every node's left subtree contains only smaller values and its right subtree contains only larger values, enabling efficient ordered operations. In a balanced BST, insertion, deletion, and search are all $O(\log h)$ where h is the height (roughly $\log n$ for a balanced tree). In the worst case (a degenerate, list-like tree caused by inserting sorted data without rebalancing), all three operations degrade to $O(n)$.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)

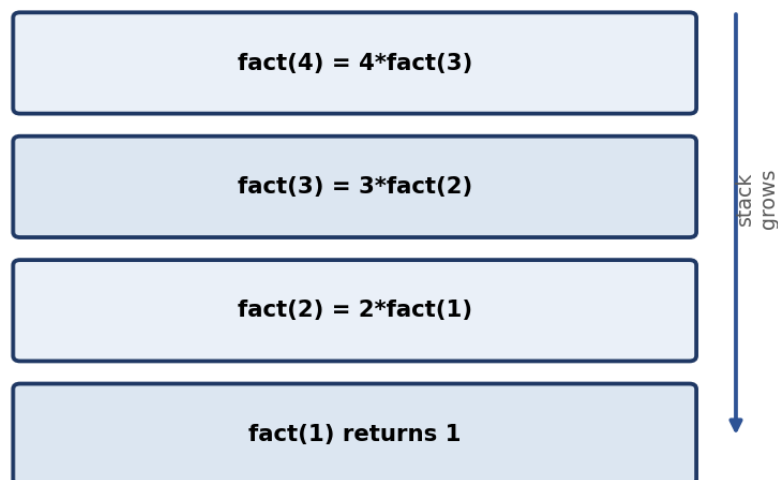


Q72. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.

Call Stack while computing factorial(4)

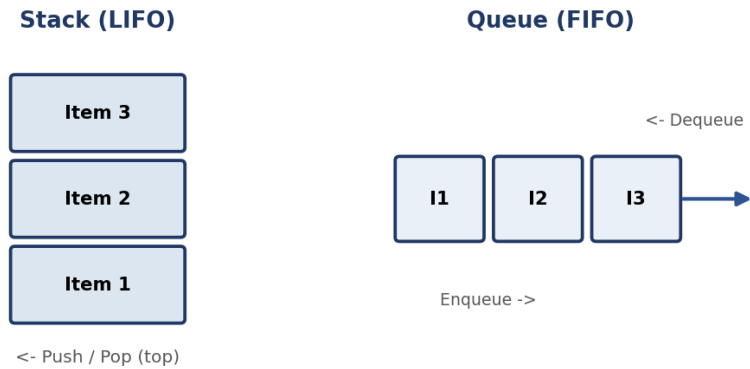


Coding Problems

Q73. Write a program to implement a stack using two queues (or vice versa).

Answer:

To implement a queue using two stacks: push new elements onto stack1; for dequeue, if stack2 is empty, pop everything from stack1 and push it onto stack2 (reversing the order), then pop from stack2 - this gives FIFO behavior with amortized $O(1)$ per operation. Implementing a stack using two queues works similarly in reverse, rearranging elements between the two queues so the most recently added element comes out first.



Q74. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average $O(n \log n)$, worst case $O(n^2)$ with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed $O(n \log n)$ in all cases, but needs $O(n)$ auxiliary space for merging.

Sorting Algorithm Complexity Comparison

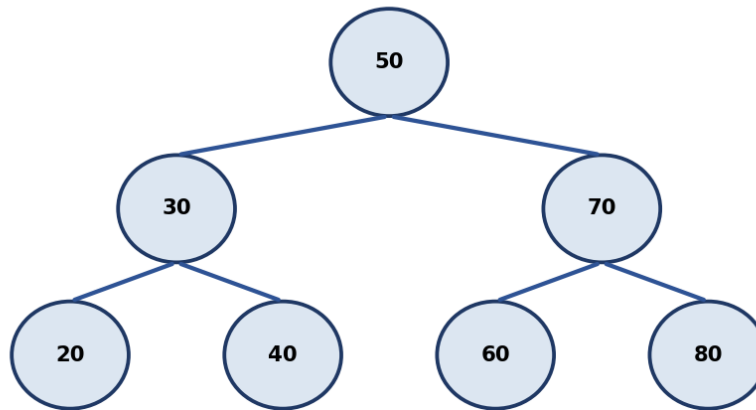
Algorithm	Best	Average	Worst	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

Q75. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in $O(h)$ time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.

Binary Search Tree (in-order = 20,30,40,50,60,70,80)



Q76. Write a program to find the maximum subarray sum (Kadane's Algorithm).

Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. $\text{current_sum} = \max(\text{arr}[i], \text{current_sum} + \text{arr}[i])$), while tracking the overall maximum seen so far. This finds the maximum subarray sum in $O(n)$ time and $O(1)$ space.

Q77. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

Use a hashmap: iterate through the array once, and for each element x , check if $(\text{target} - x)$ already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in $O(n)$ time and $O(n)$ space, instead of the $O(n^2)$ brute-force nested loop.

Q78. Write a program to check whether a binary tree is balanced.

Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving $O(n)$ time instead of the naive $O(n^2)$.

Q79. Write a program to determine if a given number is prime, and optimize it for large inputs.

Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than $\sqrt{n}$, it must also have a corresponding factor smaller than $\sqrt{n}$), skipping even numbers after checking 2, which reduces the check to roughly $O(\sqrt{n})$. For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in $O(n \log \log n)$, which is far more efficient than testing each number individually.

Q80. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for $O(1)$ lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in $O(n + m)$ time and $O(\min(n, m))$ space.

Q81. Write a program to remove duplicates from a sorted array in-place.

Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in $O(n)$ time and $O(1)$ extra space.

Q82. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic ($\text{rear} = (\text{rear} + 1) \% \text{capacity}$) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q83. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining $n-k$ elements - this three-step reversal rotates the array by k positions in-place in $O(n)$ time and $O(1)$ extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading $O(n)$ space for simpler code.)

Q84. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table $\text{dp}[i][j]$ representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If $A[i-1] == B[j-1]$, $\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$; otherwise $\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i][j-1])$. This runs in $O(n*m)$ time and space, where n and m are the string lengths.

Q85. Write a program to implement binary search on a rotated sorted array.

Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the $O(\log n)$ time complexity of standard binary search despite the rotation.

Q86. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity $O(n)$ where n is the total number of elements across all nesting levels.

Programming Language Fundamentals

Q87. Explain the difference between checked and unchecked exceptions in Java.

Answer:

Checked exceptions (e.g. `IOException`) are checked by the compiler at compile time - a method must either handle them with try-catch or declare them with 'throws', forcing the caller to acknowledge the possibility of failure.

Unchecked exceptions (e.g. `NullPointerException`, extending `RuntimeException`) are not checked at compile time and don't require explicit handling, typically representing programming errors that could occur almost anywhere.

Q88. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract `Shape` class with an abstract `area()` method but a concrete `describe()` method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a `Circle` class extending `Shape` and implementing `area()`.

Q89. What is the difference between '`==`' and '`.equals()`' when comparing objects in Java?

Answer:

'`==`' compares object references - whether two variables point to the exact same object in memory. '`.equals()`' compares the logical content/value of two objects, based on however the class has overridden it (e.g. `String`'s `.equals()` compares character sequences). Two distinct `String` objects with the same text will be '`==`' false but '`.equals()`' true, unless string interning makes them the same reference.

Q90. What is garbage collection, and how does it help prevent memory leaks?

Answer:

Garbage collection is an automatic memory management process that identifies objects no longer reachable/referenced by the running program and reclaims their memory, so the developer doesn't have to manually free it. This helps prevent memory leaks caused by forgetting to deallocate memory, though a program can still 'leak' memory logically if it keeps unnecessary references alive (e.g. objects sitting in a growing cache that's never cleared).

Q91. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. `builder.setName("X").setAge(30).build()`. To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q92. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via `malloc/new`), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q93. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: `sorted(items, key=lambda x: x.age)`. They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q94. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like `try/catch/finally`, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q95. Explain the difference between pass-by-value and pass-by-reference with an example.

Answer:

Pass-by-value copies the actual value of a variable into the function's parameter, so changes inside the function don't affect the original variable. Pass-by-reference passes a reference to the original variable's memory location, so changes inside the function do affect the original. Example: in a language with true pass-by-reference, `swap(a, b)` can actually swap the caller's variables; with pass-by-value, it would only swap local copies, leaving the caller's variables unchanged.

Q96. What is the difference between a compiler and an interpreter?

Answer:

A compiler translates the entire source program into machine code (or another target language) ahead of time, producing an executable that runs independently afterward - errors are caught before execution, and execution tends to be faster since translation work is already done. An interpreter reads and executes source code line-by-line (or statement-by-statement) at runtime without producing a standalone executable, which makes development/debugging faster but execution generally slower since translation happens repeatedly during each run.

Web & Cloud Fundamentals

Q97. Explain the difference between authentication and authorization, and how OAuth 2.0 fits in.

Answer:

Authentication verifies who a user is (e.g. checking a username/password or token), while authorization determines what an authenticated user is allowed to do (e.g. which resources/actions they can access). OAuth 2.0 is an authorization framework that lets a user grant a third-party application limited access to their resources on another service (e.g. 'Sign in with Google') without sharing their actual password, typically issuing access tokens that represent specific granted permissions.

Q98. What is containerization (e.g., Docker), and how is it different from a virtual machine?

Answer:

Containerization packages an application with its dependencies and runtime into a lightweight, portable container (e.g. via Docker) that shares the host OS kernel, making containers fast to start and resource-efficient. A virtual machine, by contrast, virtualizes an entire hardware stack including its own full guest OS, making VMs heavier and slower to start but providing stronger isolation between workloads.

Q99. Explain the concept of statelessness in REST APIs and why it matters for scalability.

Answer:

Statelessness means each request from a client to a server must contain all the information the server needs to process it, with no reliance on server-side session state stored between requests. This matters for scalability because any server instance can handle any request without needing shared session state, making it trivial to add more server instances behind a load balancer without worrying about which server 'remembers' a given client.

Q100. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Note: 19 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.